

INSTITUTE OF SPACE TECHNOLOGY
KICSIT, Kahuta Campus



FoodAI: AI-Based Android Application

Practical Implementation Report

Mini Project

Submitted By

Safa Waseem – 242201054

Course
Mobile App Development – Practical

Semester
BSCS 5-B

Assignment # 02
AI-Based Android Application

Submitted To
Sir Uzair Hassan

Department of Computer Science
KICSIT, IST Kahuta Campus

October 30, 2026

Contents

1 Introduction **1**

1.1 Project Overview 1

1.2 Problem Statement 1

1.3 Project Objectives 1

1.4 Project Scope 2

2 Tools and Technologies **2**

2.1 Android Studio 2

2.2 Java Programming Language 2

2.3 TensorFlow Lite 2

2.4 CameraX 2

2.5 Gradle 3

2.6 GitHub and GitHub Desktop 3

3 Repository Cloning and Analysis **3**

3.1 Instructor's Reference Repository 3

3.2 Repository Cloning Procedure 3

3.3 Understanding the Repository Structure 4

3.4 Reference Model File 4

3.5 Repository Evidence 4

3.6 Instructor's GitHub Repository 4

3.7 Reference Project in Android Studio 5

4 FoodAI Student Repository **7**

4.1 Student Repository Overview 7

4.2 Repository Structure 7

4.3 Purpose of GitHub Repository 7

5 Android Studio Project Structure **7**

5.1 FoodAI Application Structure 7

5.2 AndroidManifest.xml 8

5.3 MainActivity 8

5.4 CameraActivity 8

5.5 FoodClassifier 8

5.6 ResultActivity 9

5.7 Project Structure Screenshot 9

6 Application Implementation **9**

6.1 Application Workflow 9
6.2 Main User Interface 9
6.3 Camera Input 10
6.4 Image Preprocessing 10
6.5 Classification and Result Presentation 10

7 AI Model Integration **10**
7.1 TensorFlow Lite Model File 10
7.2 Labels File 10
7.3 Model Inference Process 11
7.4 Model Metadata 11
7.5 Importance of Model Compatibility 11

8 Build and Application Execution **12**
8.1 Development Requirements 12
8.2 Opening FoodAI in Android Studio 12
8.3 Building the Application 12
8.4 Running the Application 13
8.5 Build Verification 13

9 Testing and Results **13**
9.1 Testing Objectives 13
9.2 Test Cases 14
9.3 Discussion of Results 19

10 Source Code Documentation **20**
10.1 Introduction 20
10.2 SplashActivity 20
10.2.1 Purpose 20
10.2.2 Important Code Snippet 20
10.2.3 Explanation 21
10.3 LoginActivity 21
10.3.1 Purpose 21
10.3.2 Important Code Snippet: Input Validation 21
10.3.3 Explanation 22
10.3.4 Important Code Snippet: Login Verification 22
10.3.5 Explanation 22
10.4 MainActivity 23
10.4.1 Purpose 23

10.4.2 Important Code Snippet . . . 23
10.4.3 Explanation . . . 23
10.5 CameraActivity . . . 23
10.5.1 Purpose . . . 23
10.5.2 Important Code Snippet: Camera Permission . . . 23
10.5.3 Explanation . . . 24
10.5.4 Important Code Snippet: Camera Initialization . . . 24
10.5.5 Explanation . . . 25
10.5.6 Important Code Snippet: Gallery Image Selection . . . 25
10.5.7 Explanation . . . 26
10.5.8 Important Code Snippet: Classification Navigation . . . 26
10.5.9 Explanation . . . 27
10.6 FoodClassifier . . . 27
10.6.1 Purpose . . . 27
10.6.2 Important Code Snippet: Model and Labels . . . 27
10.6.3 Explanation . . . 28
10.6.4 Important Code Snippet: Loading the Model . . . 28
10.6.5 Explanation . . . 28
10.6.6 Important Code Snippet: Loading Labels . . . 29
10.6.7 Explanation . . . 29
10.6.8 Important Code Snippet: Image Classification . . . 29
10.6.9 Explanation . . . 30
10.6.10 Resource Management . . . 31
10.6.11 Explanation . . . 31
10.7 ResultActivity . . . 31
10.7.1 Purpose . . . 31
10.7.2 Important Code Snippet: Receiving Prediction Data . . . 31
10.7.3 Explanation . . . 32
10.7.4 Important Code Snippet: Nutrition Information . . . 32
10.7.5 Explanation . . . 33
10.8 Overall Application Workflow . . . 33
10.9 Code Documentation Summary . . . 33

11 Expected Learning Outcomes **33**
11.1 Understanding Android Studio . . . 33
11.2 Cloning and Inspecting GitHub Repositories . . . 33
11.3 Integrating AI and Machine Learning Models . . . 34
11.4 Understanding Image Processing . . . 34

11.5 Understanding TensorFlow Lite 34
11.6 Debugging and Testing 34
11.7 GitHub Repository Management 34
11.8 Implementation Explanation 34

12 README and Repository Documentation **34**
12.1 Purpose of README 34
12.2 Required README Contents 34
12.3 Repository Organization 34

13 Limitations and Future Improvements **35**
13.1 Project Limitations 35
13.2 Future Improvements 35

14 Conclusion **35**

15 References and Acknowledgements **36**

1 Introduction

1.1 Project Overview

FoodAI is an AI-based Android mini-project developed using Android Studio. The project is designed to provide a mobile application interface for food image recognition.

The application is organized into different components that are intended to manage the user interface, camera input, image classification, and result presentation.

The visible project package is:

`com.example.foodai`

The main Java classes identified in the project are:

- **MainActivity:** Main screen and application entry point.
- **CameraActivity:** Camera-related functionality.
- **FoodClassifier:** Machine learning classification logic.
- **ResultActivity:** Presentation of classification results.

The project also contains two important model-related files:

- `model.tflite`
- `labels.txt`

The purpose of these files is to support the intended food classification workflow. Their compatibility and actual prediction behaviour must be confirmed through testing.

1.2 Problem Statement

Identifying food items from images manually may require time and effort. Artificial Intelligence can help automate image recognition by processing an image through a trained machine learning model.

FoodAI aims to provide an Android-based application structure in which a user can provide a food image and obtain a classification result through an integrated TensorFlow Lite model.

1.3 Project Objectives

The main objectives of FoodAI are:

- To understand Android application development using Android Studio.
- To study and clone an instructor-provided GitHub repository.
- To develop an Android application using Java.
- To understand the integration of TensorFlow Lite.
- To work with image input and image preprocessing.
- To understand camera integration through CameraX.

- To organize application functionality into separate classes.
- To test the application and document its implementation.
- To manage and maintain the project using GitHub.

1.4 Project Scope

The scope of this mini-project includes Android application structure, user interface development, camera-related components, TensorFlow Lite model assets, image processing, classification logic, and result presentation.

The final working functionality depends on the actual implementation, correct model configuration, and successful execution on an Android device or emulator.

2 Tools and Technologies

2.1 Android Studio

Android Studio is the Integrated Development Environment (IDE) used for Android application development. It provides tools for writing code, designing user interfaces, managing project resources, building APK files, and testing applications.

In FoodAI, Android Studio is used to open the project, inspect its structure, manage Java classes, configure dependencies, and perform application builds.

2.2 Java Programming Language

Java is used to implement the application logic. Android activities can be written in Java to manage screens, user interactions, and communication between different application components.

FoodAI contains separate Java classes for the main screen, camera functionality, classification logic, and result presentation.

2.3 TensorFlow Lite

TensorFlow Lite is a framework designed to run machine learning models on mobile and embedded devices.

FoodAI contains a model file named `model.tflite`. The model is intended to perform food image classification.

For correct inference, the application must provide input data in the format expected by the model. The output tensor must also be interpreted correctly.

2.4 CameraX

CameraX is an Android Jetpack library that simplifies camera development.

The project configuration includes the following CameraX components:

- `camera-core`
- `camera-camera2`

- `camera-lifecycle`
- `camera-view`

These libraries provide functionality for camera preview, camera control, lifecycle management, and image capture.

2.5 Gradle

Gradle is the build automation system used by Android Studio. It manages project configuration, dependencies, compilation, and APK generation.

The visible FoodAI configuration includes CameraX version 1.3.4 and TensorFlow Lite version 2.14.0. The final dependency versions should be confirmed from the submitted Gradle files.

2.6 GitHub and GitHub Desktop

GitHub is used to store and manage the project repository online. GitHub Desktop provides a graphical interface for cloning repositories, viewing changes, committing files, and pushing updates.

These tools help maintain project versions and make the source code available for review.

3 Repository Cloning and Analysis

3.1 Instructor's Reference Repository

The instructor-provided reference repository is:

https://github.com/Uzair390-Del/NumberGuessingAppTflite

This repository demonstrates an Android application that uses TensorFlow Lite. It was studied as a reference for understanding Android project organization and machine learning model integration.

It is important to note that this is a number-guessing application, not a food classification application. Its purpose in this project is to provide a reference for learning the Android and TensorFlow Lite workflow.

3.2 Repository Cloning Procedure

The repository can be cloned using GitHub Desktop. The procedure is:

1. Open GitHub Desktop.
2. Select **File** from the menu.
3. Select **Clone Repository**.
4. Open the URL tab.
5. Enter the instructor's GitHub repository link.
6. Select a suitable local folder.
7. Click the Clone button.

8. Wait for the repository to download.
9. Open the cloned project folder in Android Studio.

The repository can also be cloned using the following Git command:

```
git clone https://github.com/Uzair390-Del/NumberGuessingAppTflite.git
```

3.3 Understanding the Repository Structure

After cloning, the project structure was inspected to understand the purpose of its main files and folders.

Common Android project components include:

- `app`: Main Android application module.
- `src`: Source code and application resources.
- `MainActivity.java`: Main activity of the reference application.
- `assets`: Folder for model and other asset files.
- `res`: Application resources such as layouts and images.
- `build.gradle`: Build and dependency configuration.
- `settings.gradle`: Project module settings.
- `gradlew`: Gradle wrapper script.

3.4 Reference Model File

The reference repository contains a model file named `linear.tflite`. This file belongs to the reference application.

FoodAI contains its own model file named `model.tflite`. The two model files should not be treated as interchangeable because they may have different input and output specifications.

3.5 Repository Evidence

The following screenshots provide evidence of the repository inspection and cloning process.

3.6 Instructor's GitHub Repository

The instructor provided a reference GitHub repository to help understand the structure and implementation of an Android application using TensorFlow Lite.

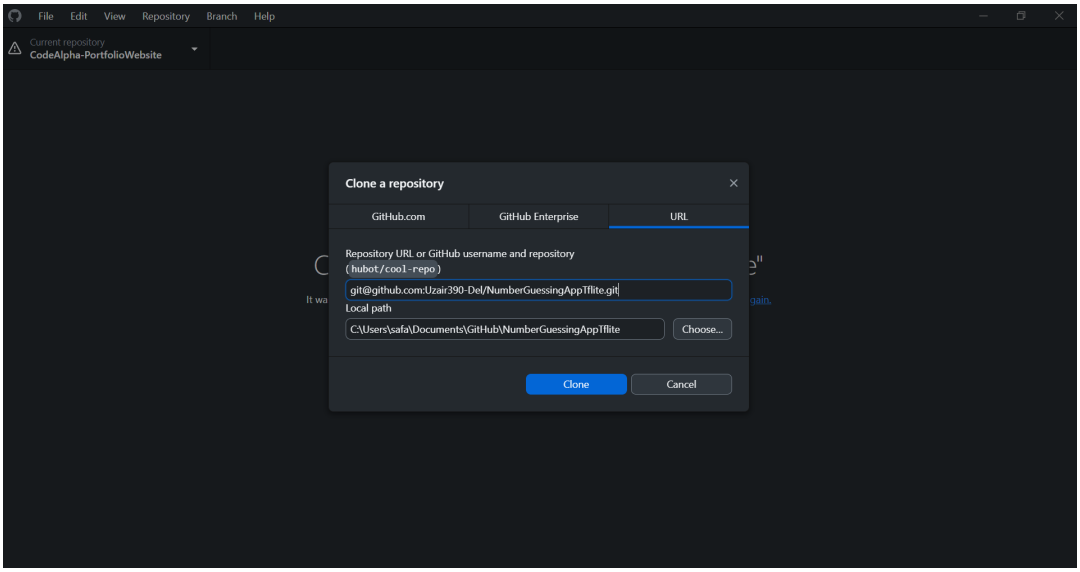


Figure 3.1: Instructor's GitHub Repository

3.7 Reference Project in Android Studio

After cloning, the reference project was opened in Android Studio. Its folder structure, source files, resources, and Gradle configuration were examined to understand the Android project organization and TensorFlow Lite integration.

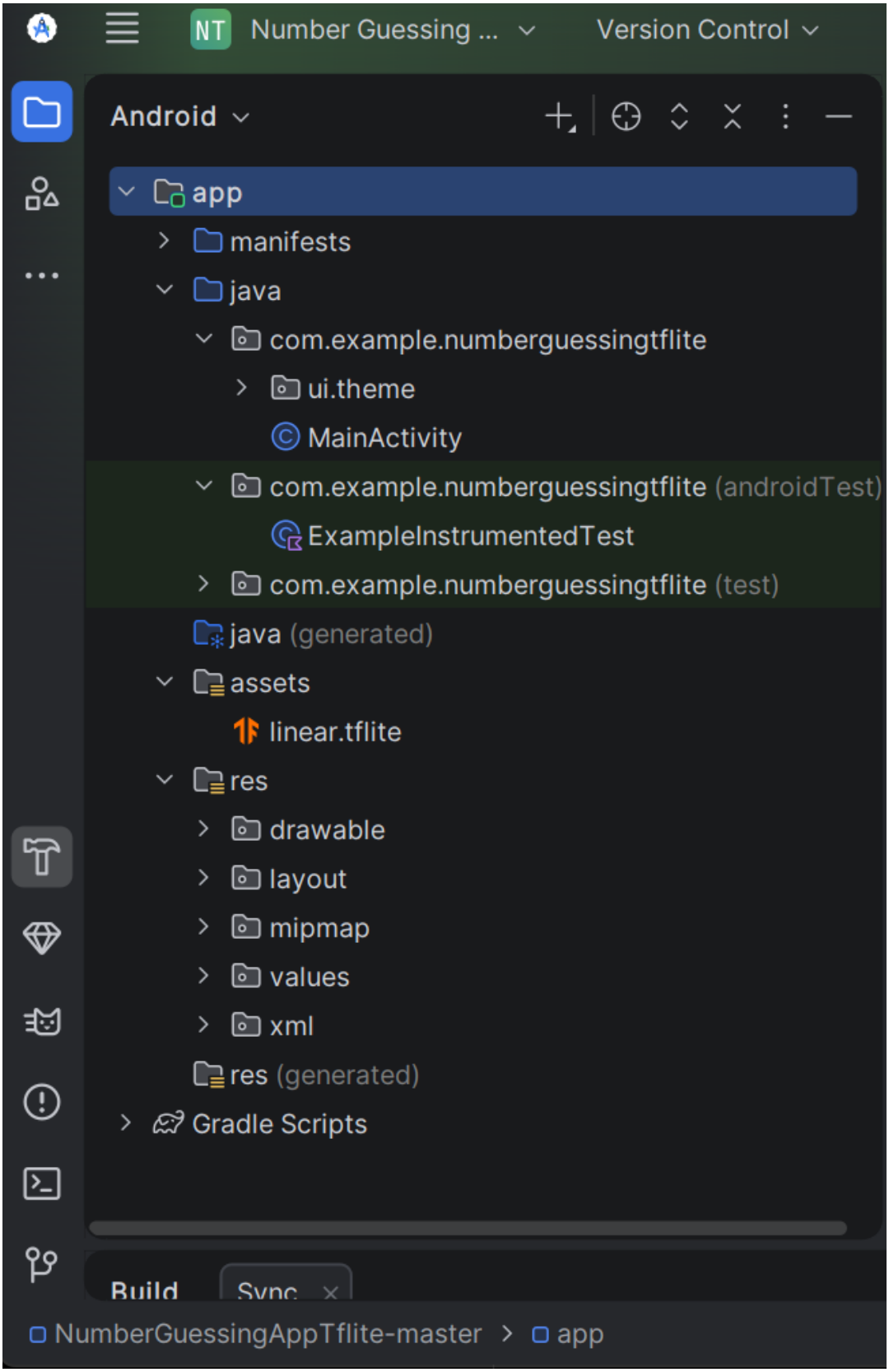


Figure 3.2: Reference Project Opened in Android Studio

4 FoodAI Student Repository

4.1 Student Repository Overview

The student project repository is named FoodAI and is available at:

https://github.com/safa699/FoodAI

This repository contains the Android Studio project prepared for the FoodAI mini-project. It is separate from the instructor's reference repository.

4.2 Repository Structure

The visible repository contains the following important files and folders:

- `.idea`
- `app`
- `gradle`
- `.gitignore`
- `build.gradle`
- `gradle.properties`
- `gradlew`
- `gradlew.bat`
- `settings.gradle`

The `app` folder contains the main Android application module. The Gradle files provide build configuration and dependency management.

4.3 Purpose of GitHub Repository

GitHub provides a central location for storing and managing the project source code. It supports version control and allows the developer to track changes made during development.

The repository also makes it easier for the instructor to inspect the source code and project documentation.

If the repository is private, access should be granted to the instructor before submission.

5 Android Studio Project Structure

5.1 FoodAI Application Structure

The FoodAI application is organized into different components. The visible project files include the Android manifest, Java activity classes, and TensorFlow Lite assets.

The main components are:

- `AndroidManifest.xml`
- `MainActivity.java`
- `CameraActivity.java`
- `FoodClassifier.java`
- `ResultActivity.java`
- `model.tflite`
- `labels.txt`

5.2 AndroidManifest.xml

The Android manifest file contains important application information. It can declare application activities, permissions, and other configuration details.

For FoodAI, camera-related permissions must be declared if the application uses the device camera.

5.3 MainActivity

MainActivity acts as the main entry point of the application. It is responsible for displaying the initial screen and providing access to the application's main workflow.

The exact interface and

navigation behaviour depend on the implemented Java code and layout resources.

5.4 CameraActivity

CameraActivity is intended to manage camera-related operations. It may provide camera preview and image capture functionality through CameraX.

Camera permission handling and lifecycle management are important for reliable camera operation.

5.5 FoodClassifier

FoodClassifier is intended to handle the machine learning classification process.

Its responsibilities may include:

- Loading the TensorFlow Lite model.
- Preparing the input image.
- Resizing the image.
- Converting image pixels into the required tensor format.
- Running model inference.
- Interpreting the prediction output.

5.6 ResultActivity

ResultActivity is intended to display the classification result. It receives the output from the classification workflow and presents the relevant information to the user.

5.7 Project Structure Screenshot

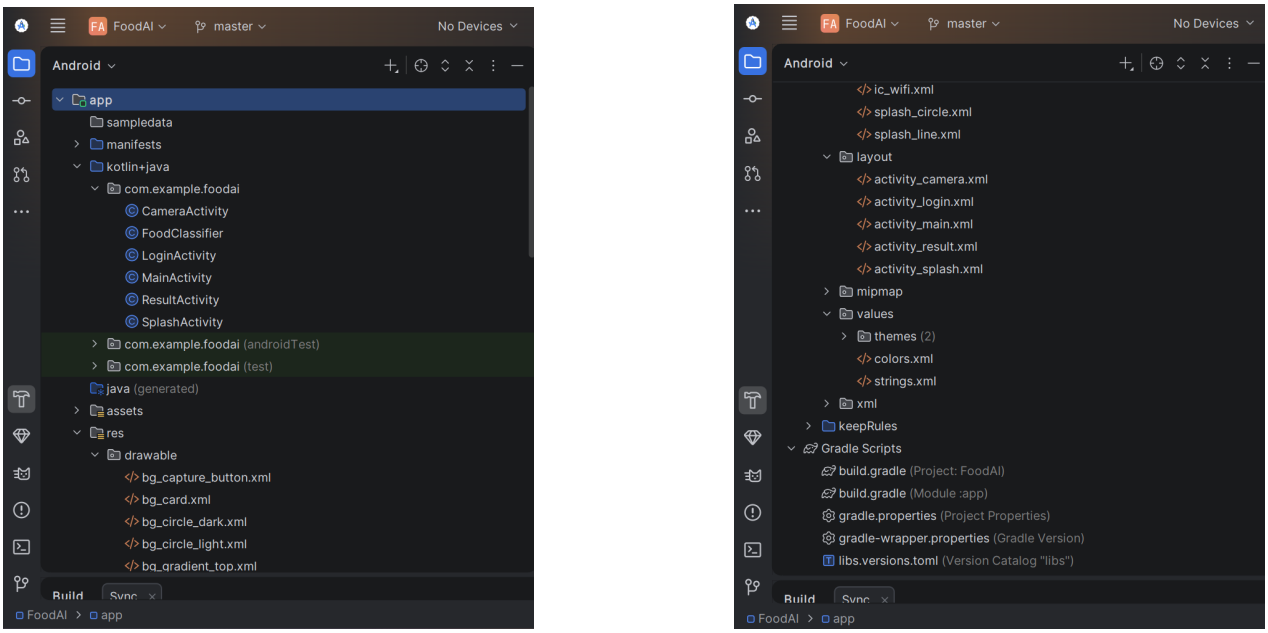


Figure 5.1: FoodAI Project Structure in Android Studio

6 Application Implementation

6.1 Application Workflow

The intended FoodAI workflow consists of the following stages:

1. The user opens the FoodAI application.
2. The main screen provides access to the image input process.
3. The user captures or selects a food image, if this functionality is implemented.
4. The image is passed to the classification component.
5. The image is prepared according to the model's input requirements.
6. The TensorFlow Lite model processes the image.
7. The output is interpreted using the corresponding class labels.
8. The result screen presents the classification output.

6.2 Main User Interface

The main user interface is the starting point of the application. It should provide a clear way for the user to begin the image recognition process.

The interface should be tested to ensure that buttons and navigation controls perform their intended actions.

6.3 Camera Input

CameraX dependencies are included in the project configuration. Camera functionality requires appropriate permission handling and camera lifecycle management.

The camera screen should be tested on a physical device or emulator that supports camera access.

6.4 Image Preprocessing

Image preprocessing prepares the input image for the machine learning model.

Depending on the model specifications, preprocessing may involve:

- Resizing the image to the required dimensions.
- Converting the image into pixel values.
- Arranging pixel values in the correct tensor order.
- Normalizing pixel values when required.
- Converting data into the model's expected input type.

The exact preprocessing method must match the model's input specifications.

6.5 Classification and Result Presentation

After preprocessing, the image data is passed to the TensorFlow Lite interpreter. The interpreter executes the model and produces output values.

The application must map the output values to the correct labels. The predicted class can then be displayed through the result screen.

Actual classification results should only be reported after testing with suitable food images.

7 AI Model Integration

7.1 TensorFlow Lite Model File

FoodAI contains a TensorFlow Lite model file named:

`model.tflite`

The file is located in the application's assets directory. It is intended to provide the machine learning functionality for food image classification.

The model must be compatible with the input image format and output processing implemented in the application.

7.2 Labels File

The project also contains:

`labels.txt`

This file is intended to contain the class labels used by the classification model.

The order of labels must match the order of the model's output classes. Incorrect label ordering may cause the application to display an incorrect class name.

7.3 Model Inference Process

The general TensorFlow Lite inference process consists of the following steps:

1. Load the model file from the application assets.
2. Create or initialize the TensorFlow Lite interpreter.
3. Read the input image.
4. Resize and preprocess the image.
5. Convert the image into the required input tensor format.
6. Pass the input tensor to the interpreter.
7. Execute the model.
8. Read the output tensor.
9. Identify the predicted class.
10. Match the prediction with the corresponding label.
11. Display the result to the user.

7.4 Model Metadata

During model inspection, the Android Studio model viewer displayed the message:

No metadata found in this model

This message does not necessarily indicate that the model is invalid. However, it means that the developer should verify the model's input dimensions, tensor data type, output dimensions, and preprocessing requirements separately.

7.5 Importance of Model Compatibility

A TensorFlow Lite model can only produce meaningful results when the application supplies correctly formatted input data.

The developer must verify:

- Input tensor dimensions.
- Input data type.
- Image resizing requirements.

- Pixel normalization requirements.
- Output tensor dimensions.
- Class label ordering.

8 Build and Application Execution

8.1 Development Requirements

The following tools and components are required to open and run the project:

- Android Studio.
- Compatible Java Development Kit.
- Android SDK.
- Gradle wrapper.
- Required project dependencies.
- Android emulator or physical Android device.
- Internet connection for downloading dependencies when required.

8.2 Opening FoodAI in Android Studio

The following steps are used to open the project:

1. Launch Android Studio.
2. Select the **Open** option.
3. Navigate to the FoodAI project folder.
4. Select the project root directory.
5. Wait for Android Studio to load the project.
6. Allow Gradle synchronization to complete.

8.3 Building the Application

After opening the project, the developer should allow Android Studio to synchronize the Gradle configuration.

The application can then be built using the Build menu.

A debug build can also be generated through the Windows terminal:

```
gradlew.bat assembleDebug
```

This command should be executed from the root directory of the FoodAI project.

8.4 Running the Application

After a successful build, the application can be tested using an Android emulator or a physical Android device.

The general procedure is:

1. Connect an Android device or start an emulator.
2. Select the appropriate device in Android Studio.
3. Select the FoodAI run configuration.
4. Click the Run button.
5. Wait for the application to install.
6. Open the application.
7. Test the available features.

8.5 Build Verification

A successful build should be confirmed through Android Studio's Build output. The final report should include a screenshot showing the actual successful FoodAI build.

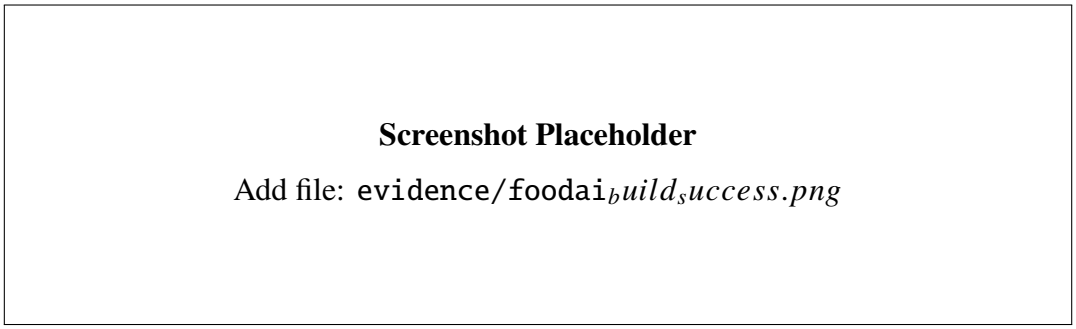


Figure 8.1: FoodAI successful build in Android Studio

9 Testing and Results

9.1 Testing Objectives

Testing is performed to evaluate whether the application components work according to their intended purpose.

The main testing objectives include:

- Verifying project synchronization.
- Checking successful compilation.
- Testing application installation.
- Opening the main application screen.
- Checking camera permission handling.

- Testing image input.
- Verifying model loading.
- Checking model inference.
- Verifying label mapping.
- Checking result presentation.

9.2 Test Cases

The following test cases can be used to evaluate the FoodAI application.

No.	Test Case	Expected Result	Status
1	Repository cloning	Repository is available locally	Verify
2	Project opening	Project opens in Android Studio	Verify
3	Gradle synchronization	Required dependencies synchronize	Verify
4	Application build	APK is generated successfully	Verify
5	Application launch	Main screen opens	Verify
6	Camera permission	Camera permission is handled correctly	Verify
7	Camera preview	Camera preview appears	Verify
8	Image processing	Image is prepared for model input	Verify
9	Model loading	TensorFlow Lite model loads successfully	Verify
10	Model inference	Model produces an output	Verify
11	Label mapping	Output is matched with the correct label	Verify
12	Result display	Classification result is presented	Verify

The status column must be updated according to actual testing. Do not mark a test as Passed unless it has been performed successfully.

FoodAI Application Screenshots

1. **Splash Screen:**

   The splash screen is the first interface displayed when FoodAI starts. It introduces the application and provides a starting point for the user journey.

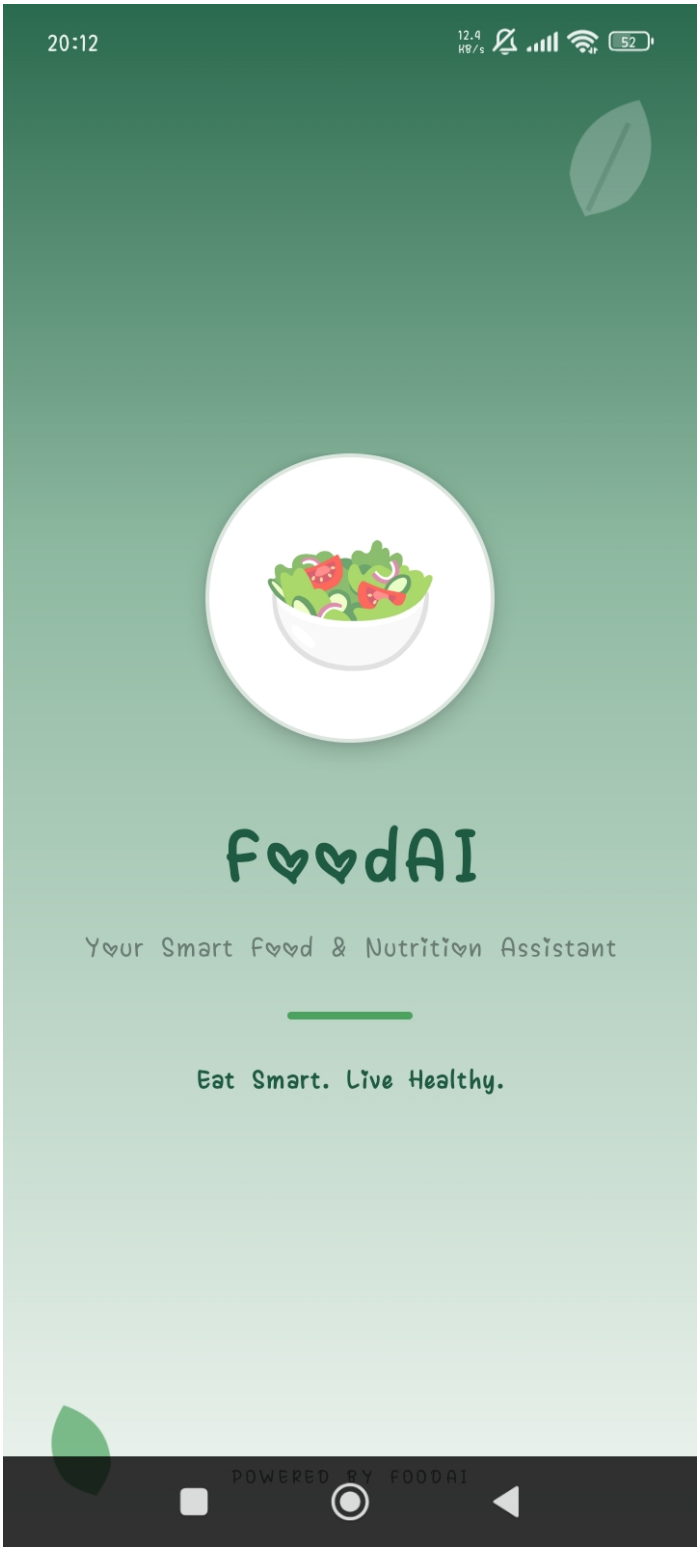


Figure 9.1: FoodAI Application Splash Screen

2. **Login and Sign-up Screens:**

   The login screen allows existing users to enter their credentials, while the sign-up screen provides an interface for new users to register, if authentication is implemented.

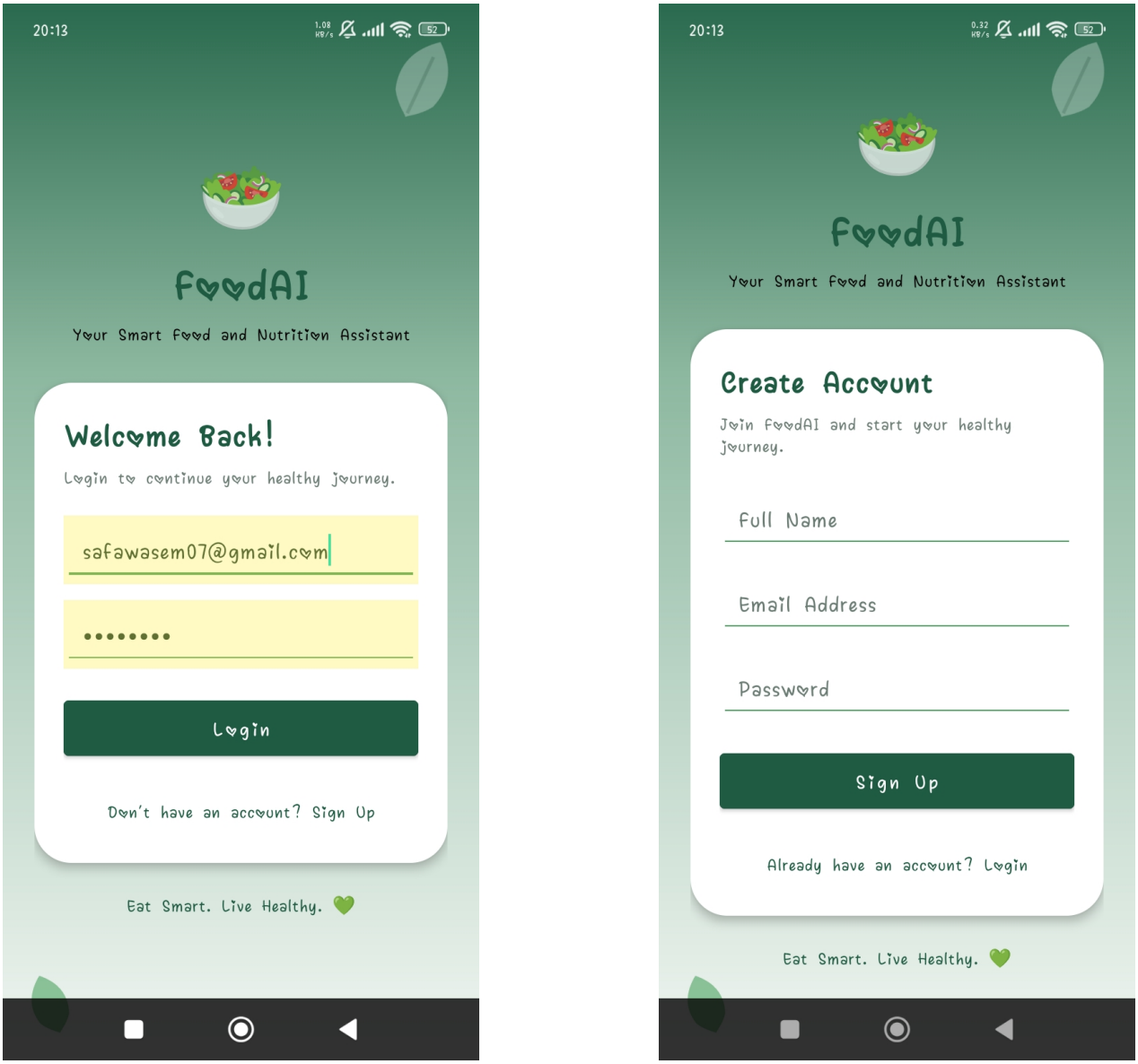


Figure 9.2: FoodAI Login and Sign-up Interfaces

3. **Home Screen and Navigation to Food Scanning:**

   The home screen introduces the main functionality of FoodAI. When the user clicks the scanning button, the application opens the food-scanning interface.

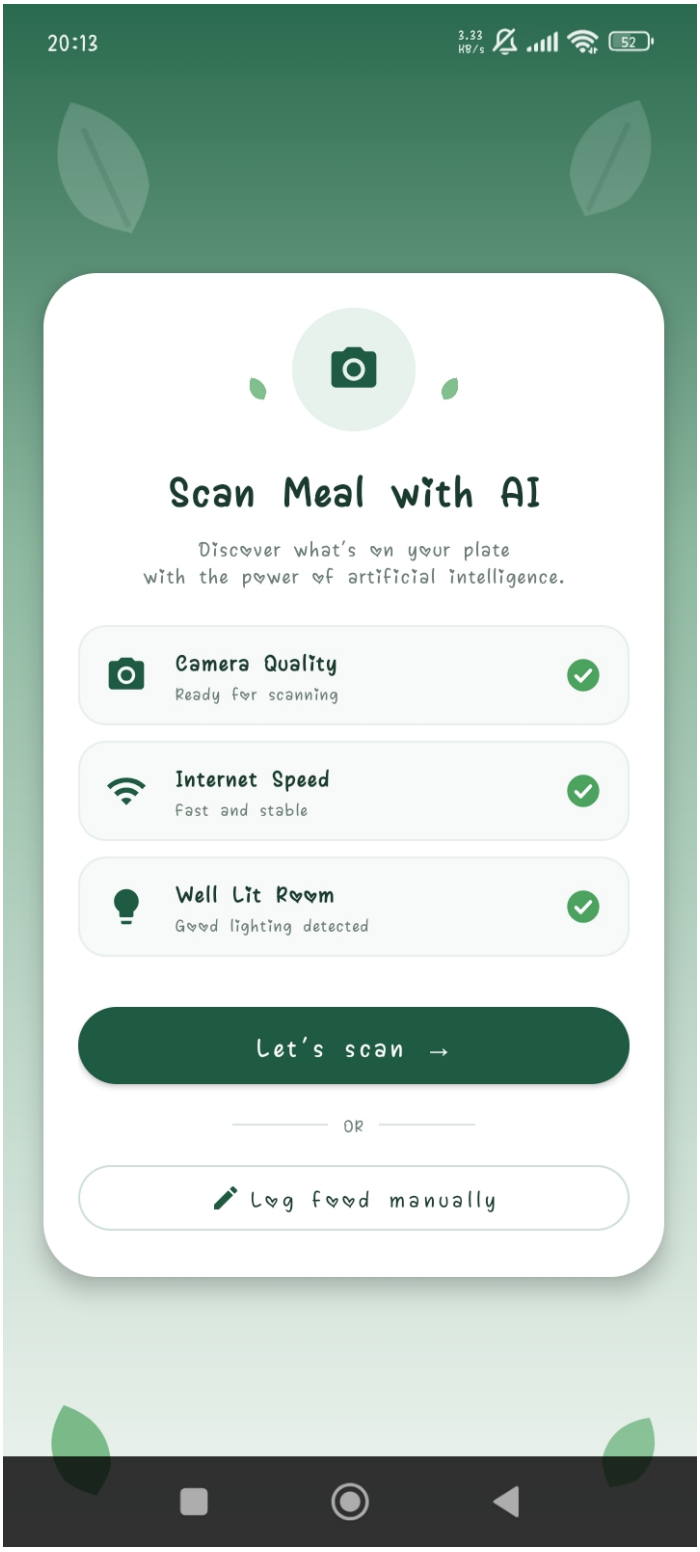


Figure 9.3: Goto Food Image Scanning Interface

4. **Food-Scanning Screen:**

   This screen allows users to choose a food image from the gallery or capture one using the camera. The selected image is then used for the food recognition process.

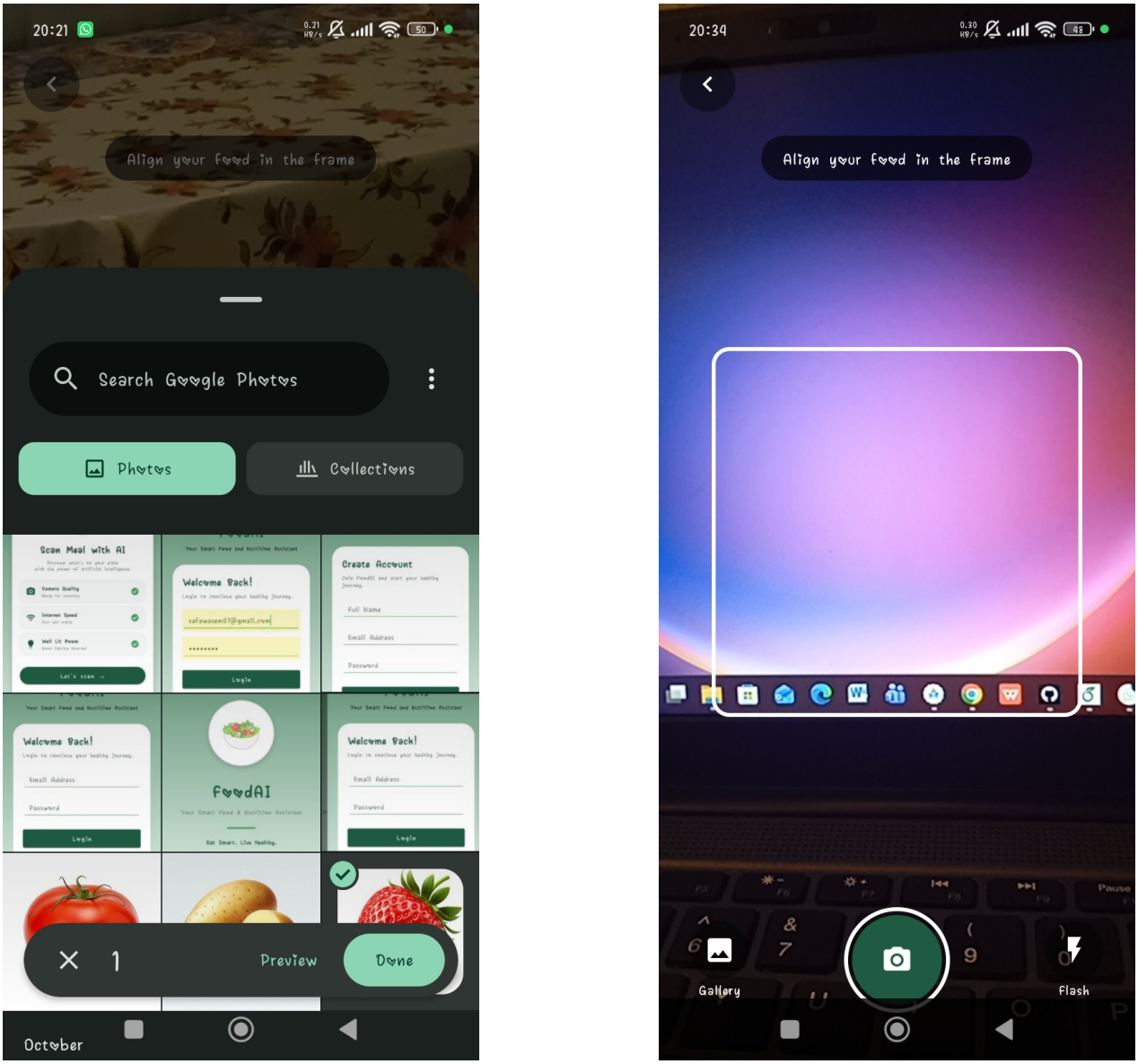


Figure 9.4: Food Image Selected from Gallery or Camera

5. **Selected or Captured Food Image:**

   The selected or captured image is displayed so the user can review it before analysis. The image may then be prepared according to the input requirements of the AI model.

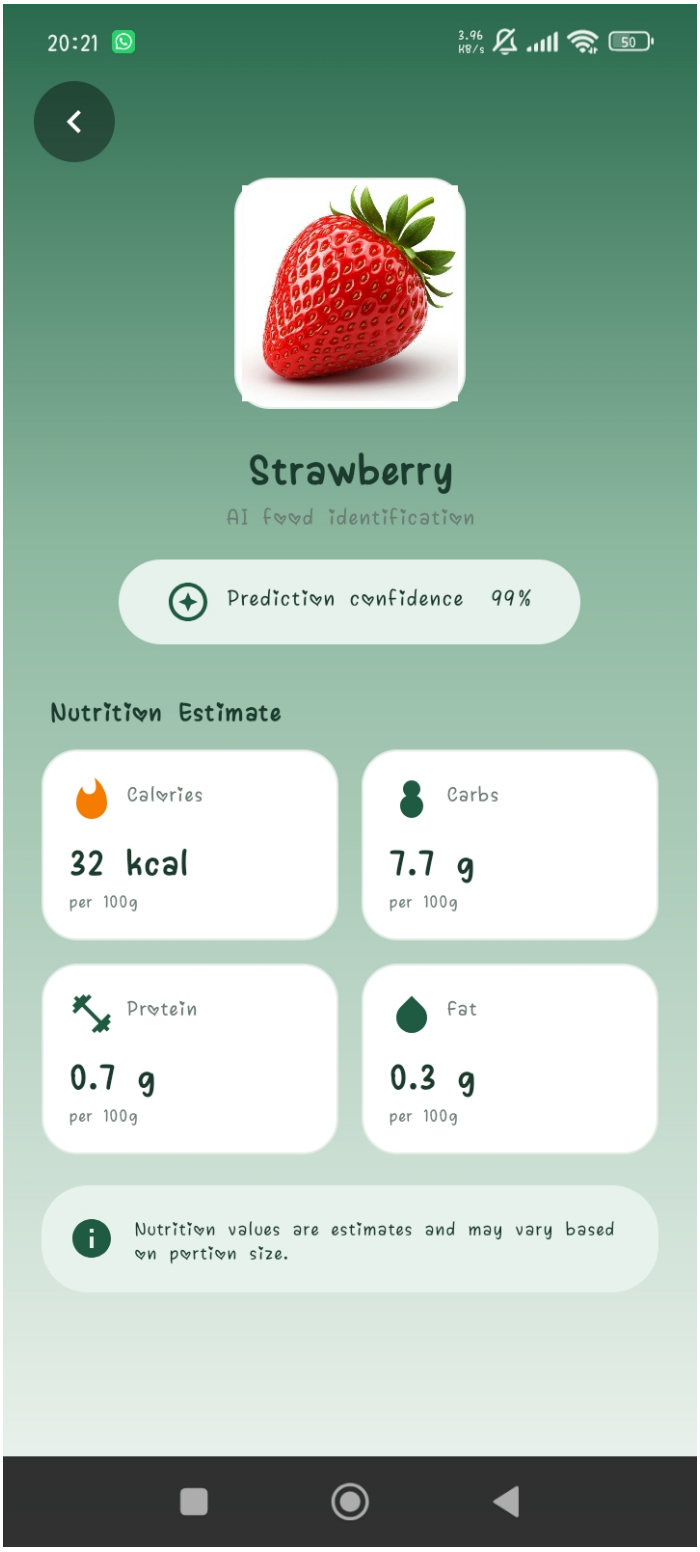


Figure 9.5: FoodAI AI Prediction Result

9.3 Discussion of Results

The results of the application should be evaluated after performing the test cases.

The developer should confirm whether the application opens correctly, whether image input works, whether the TensorFlow Lite model loads, and whether the displayed classification label matches the test image.

Prediction accuracy should not be claimed unless it has been measured using an appropriate test dataset.

10 Source Code Documentation

10.1 Introduction

The FoodAI application is developed using Java in Android Studio. The source code is divided into separate classes to manage different application functions, including splash screen navigation, user login, main screen navigation, camera operations, machine learning classification, and result presentation.

The project contains the following Java files:

- SplashActivity.java
- LoginActivity.java
- MainActivity.java
- CameraActivity.java
- FoodClassifier.java
- ResultActivity.java

The following sections describe the important code snippets from each class.

10.2 SplashActivity

10.2.1 Purpose

SplashActivity displays the application splash screen when FoodAI starts. After a delay of 2.5 seconds, the application navigates to LoginActivity.

10.2.2 Important Code Snippet

Listing 10.1: SplashActivity: Splash Screen and Navigation

```
@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_splash);

    new Handler(Looper.getMainLooper()).postDelayed(() -> {
        Intent intent = new Intent(
            SplashActivity.this,
            LoginActivity.class
        );
        startActivity(intent);
        finish();
    }, 2500);
}
```

10.2.3 Explanation

- The `onCreate()` method initializes the splash screen.
- `setContentView()` loads the splash screen layout.
- Handler and postDelayed() create a delay of 2500 milliseconds.
- An Intent is used to open LoginActivity.
- The finish() method closes SplashActivity after navigation.

10.3 LoginActivity

10.3.1 Purpose

LoginActivity provides login and sign-up functionality. It collects the user's name, email, and password and performs basic input validation. The demo account information is stored locally using SharedPreferences.

10.3.2 Important Code Snippet: Input Validation

Listing 10.2: LoginActivity: Sign-Up Validation

```
private void signUp() {
    String name = nameInput.getText().toString().trim();
    String email = emailInput.getText().toString()
            .trim().toLowerCase();
    String password = passwordInput.getText().toString();

    if (name.isEmpty()) {
        nameInput.setError("Enter your name");
        nameInput.requestFocus();
        return;
    }

    if (!Patterns.EMAIL_ADDRESS.matcher(email).matches()) {
        emailInput.setError("Enter a valid email");
        emailInput.requestFocus();
        return;
    }

    if (password.length() < 6) {
        passwordInput.setError(
            "Password must be at least 6 characters");
        passwordInput.requestFocus();
        return;
    }

    preferences.edit()
            .putString("name", name)
            .putString("email", email)
```

```
            .putString("password", password)
            .apply();

    openMainScreen();
}
```

10.3.3 Explanation

- The method retrieves the name, email, and password entered by the user.
- It checks whether the name field is empty.
- Patterns.EMAIL_ADDRESS validates the email format.
- The password must contain at least six characters.
- SharedPreferences stores the demo account information locally.
- After successful registration, openMainScreen() opens MainActivity.

10.3.4 Important Code Snippet: Login Verification

Listing 10.3: LoginActivity: Login Verification

```
private void login() {
    String email = emailInput.getText().toString()
            .trim().toLowerCase();
    String password = passwordInput.getText().toString();

    String savedEmail = preferences.getString("email", "");
    String savedPassword = preferences.getString("password", "");

    if (email.equals(savedEmail)
            && password.equals(savedPassword)
            && !savedEmail.isEmpty()) {

        openMainScreen();

    } else {
        Toast.makeText(this,
            "Account not found or password incorrect.",
            Toast.LENGTH_LONG).show();
    }
}
```

10.3.5 Explanation

This method compares the entered email and password with the information stored in SharedPreferences. If both values match, the application opens MainActivity. Otherwise, an error message is displayed.

Note: This is a local demonstration login system. It is not a secure server-based authentication system.

10.4 MainActivity

10.4.1 Purpose

MainActivity represents the main screen of the FoodAI application. It provides navigation to the camera screen and includes a manual logging button.

10.4.2 Important Code Snippet

Listing 10.4: MainActivity: Button Click Listeners

```
@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_main);

    Button btnScan = findViewById(R.id.btnScan);
    Button btnManual = findViewById(R.id.btnManual);

    btnScan.setOnClickListener(v ->
        startActivity(new Intent(
            MainActivity.this,
            CameraActivity.class
        ))
    );

    btnManual.setOnClickListener(v ->
        Toast.makeText(this,
            "Manual log jald aayega",
            Toast.LENGTH_SHORT).show()
    );
}
```

10.4.3 Explanation

- The main screen layout is loaded using setContentView().
- findViewById() connects Java code with XML buttons.
- The Scan button opens CameraActivity through an Intent.
- The Manual button currently displays a message using Toast.

10.5 CameraActivity

10.5.1 Purpose

CameraActivity manages camera operations and image selection. It uses CameraX to display the camera preview and capture images. It also allows users to select an image from the device gallery.

10.5.2 Important Code Snippet: Camera Permission

Listing 10.5: CameraActivity: Camera Permission Handling

```
if (ContextCompat.checkSelfPermission(
        this, Manifest.permission.CAMERA)
        == PackageManager.PERMISSION_GRANTED) {

    startCamera();

} else {
    permissionLauncher.launch(
        Manifest.permission.CAMERA
    );
}
```

10.5.3 Explanation

- checkSelfPermission() checks whether camera permission is granted.
- If permission is available, startCamera() initializes the camera.
- Otherwise, permissionLauncher requests camera permission.
- If permission is denied, the application displays a message.

10.5.4 Important Code Snippet: Camera Initialization

Listing 10.6: CameraActivity: CameraX Initialization

```
private void startCamera() {
    ListenableFuture<ProcessCameraProvider> future =
            ProcessCameraProvider.getInstance(this);

    future.addListener(() -> {
        try {
            ProcessCameraProvider provider = future.get();

            Preview preview = new Preview.Builder().build();

            preview.setSurfaceProvider(
                previewView.getSurfaceProvider()
            );

            imageCapture = new ImageCapture.Builder().build();

            provider.unbindAll();

            camera = provider.bindToLifecycle(
                this,
                CameraSelector.DEFAULT_BACK_CAMERA,
                preview,
                imageCapture
            );
```

```
        } catch (ExecutionException | InterruptedException e) {
            Toast.makeText(this,
                "Camera error: " + e.getMessage(),
                Toast.LENGTH_LONG).show();
        }
    }, ContextCompat.getMainExecutor(this));
}
```

10.5.5 Explanation

- ProcessCameraProvider provides access to the camera.
- Preview creates the live camera preview.
- ImageCapture prepares the camera for taking photographs.
- bindToLifecycle() connects camera operations with the activity lifecycle.
- The default back camera is selected for image capture.

10.5.6 Important Code Snippet: Gallery Image Selection

Listing 10.7: CameraActivity: Gallery Image Selection

```
private void onImagePicked(Uri uri) {
    if (uri == null) {
        return;
    }

    Bitmap bitmap;

    try (InputStream inputStream =
            getContentResolver().openInputStream(uri)) {

        if (inputStream == null) {
            return;
        }

        bitmap = BitmapFactory.decodeStream(inputStream);

    } catch (IOException e) {
        Toast.makeText(this,
            "Image open error: " + e.getMessage(),
            Toast.LENGTH_LONG).show();
        return;
    }

    if (bitmap != null) {
        classifyAndShowResult(bitmap, null, uri.toString());
    }
```

```
}
```

10.5.7 Explanation

This method receives the selected image URI from the gallery. The ContentResolver opens the image stream, and BitmapFactory converts the image into a Bitmap object. The image is then sent to the common classification method.

10.5.8 Important Code Snippet: Classification Navigation

Listing 10.8: CameraActivity: Calling FoodClassifier

```
private void classifyAndShowResult(
        Bitmap bitmap, String imagePath, String imageUri) {

    FoodClassifier classifier = null;

    try {
        classifier = new FoodClassifier(this);

        FoodClassifier.Result result =
                classifier.classify(bitmap);

        Intent intent = new Intent(
            CameraActivity.this,
            ResultActivity.class
        );

        intent.putExtra(ResultActivity.EXTRA_LABEL,
                result.label);

        intent.putExtra(ResultActivity.EXTRA_CONFIDENCE,
                result.confidence);

        if (imagePath != null) {
            intent.putExtra(
                ResultActivity.EXTRA_IMAGE_PATH,
                imagePath
            );
        }

        if (imageUri != null) {
            intent.putExtra(
                ResultActivity.EXTRA_IMAGE_URI,
                imageUri
            );
        }

        startActivity(intent);
```

```
    } catch (Exception e) {
        Toast.makeText(this,
            "Prediction error: " + e.getMessage(),
            Toast.LENGTH_LONG).show();

    } finally {
        if (classifier != null) {
            classifier.close();
        }

        if (!bitmap.isRecycled()) {
            bitmap.recycle();
        }
    }
}
```

10.5.9 Explanation

- A FoodClassifier object is created to perform prediction.
- The classify() method processes the selected Bitmap image.
- The predicted label and confidence are received.
- Intent passes the prediction information to ResultActivity.
- The image path or URI is also passed when available.
- The classifier is closed after processing to release resources.

10.6 FoodClassifier

10.6.1 Purpose

FoodClassifier is the main machine learning component of FoodAI. It uses the TensorFlow Lite Interpreter to load and execute the food classification model.

The class reads model.tflite and labels.txt from the assets folder, prepares the input image, executes inference, and identifies the class with the highest output probability.

10.6.2 Important Code Snippet: Model and Labels

Listing 10.9: FoodClassifier: Model Configuration

```
private static final String MODEL_FILE = "model.tflite";
private static final String LABEL_FILE = "labels.txt";
private static final int INPUT_SIZE = 224;

private final Interpreter interpreter;
private final List<String> labels;

public FoodClassifier(Context context) throws IOException {
```

```
    interpreter = new Interpreter(loadModel(context));
    labels = loadLabels(context);
}
```

10.6.3 Explanation

- MODEL_FILE defines the TensorFlow Lite model filename.
- LABEL_FILE specifies the file containing food class names.
- INPUT_SIZE defines the image dimensions as 224 by 224 pixels.
- Interpreter executes the machine learning model.
- The constructor loads both the model and its labels.

10.6.4 Important Code Snippet: Loading the Model

Listing 10.10: FoodClassifier: Loading TensorFlow Lite Model

```
private ByteBuffer loadModel(Context context)
        throws IOException {

    try (InputStream in =
            context.getAssets().open(MODEL_FILE)) {

        ByteArrayOutputStream out =
                new ByteArrayOutputStream();

        byte[] chunk = new byte[8192];
        int n;

        while ((n = in.read(chunk)) != -1) {
            out.write(chunk, 0, n);
        }

        byte[] bytes = out.toByteArray();

        ByteBuffer buffer =
                ByteBuffer.allocateDirect(bytes.length);

        buffer.order(ByteOrder.nativeOrder());
        buffer.put(bytes);
        buffer.rewind();

        return buffer;
    }
}
```

10.6.5 Explanation

- getAssets().open() accesses the model file in the assets folder.

- InputStream reads the model data.
- ByteArrayOutputStream collects the model bytes.
- ByteBuffer.allocateDirect() creates a direct memory buffer.
- The buffer is returned for TensorFlow Lite Interpreter initialization.

10.6.6 Important Code Snippet: Loading Labels

Listing 10.11: FoodClassifier: Reading Food Labels

```
private List<String> loadLabels(Context context)
        throws IOException {

    List<String> list = new ArrayList<>();

    try (BufferedReader reader = new BufferedReader(
            new InputStreamReader(
                context.getAssets().open(LABEL_FILE)))) {

        String line;

        while ((line = reader.readLine()) != null) {
            if (!line.trim().isEmpty()) {
                list.add(line.trim());
            }
        }
    }

    return list;
}
```

10.6.7 Explanation

This method reads labels.txt line by line and stores each non-empty label in an ArrayList. These labels are used to convert the model's output index into a readable food name.

10.6.8 Important Code Snippet: Image Classification

Listing 10.12: FoodClassifier: Preparing Image Input

```
public Result classify(Bitmap source) {

    Bitmap bitmap = Bitmap.createScaledBitmap(
        source, INPUT_SIZE, INPUT_SIZE, true
    );

    ByteBuffer input = ByteBuffer.allocateDirect(
        4 * INPUT_SIZE * INPUT_SIZE * 3
    );
```

```
    input.order(ByteOrder.nativeOrder());

    int[] pixels = new int[
        INPUT_SIZE * INPUT_SIZE
    ];

    bitmap.getPixels(
        pixels, 0, INPUT_SIZE,
        0, 0, INPUT_SIZE, INPUT_SIZE
    );

    for (int pixel : pixels) {
        float r = (pixel >> 16) & 0xFF;
        float g = (pixel >> 8) & 0xFF;
        float b = pixel & 0xFF;

        input.putFloat(r);
        input.putFloat(g);
        input.putFloat(b);
    }

    float[][] output = new float[1][labels.size()];

    interpreter.run(input, output);

    int best = 0;

    for (int i = 1; i < output[0].length; i++) {
        if (output[0][i] > output[0][best]) {
            best = i;
        }
    }

    return new Result(labels.get(best), output[0][best]);
}
```

10.6.9 Explanation

- The input image is resized to 224 by 224 pixels.
- A ByteBuffer is created to store RGB pixel values.
- getPixels() extracts the image pixel data.
- Red, green, and blue values are extracted from each pixel.
- interpreter.run() executes the TensorFlow Lite model.
- The output array contains the prediction scores.
- A loop identifies the class with the highest score.

- The method returns the predicted label and confidence.

10.6.10 Resource Management

Listing 10.13: FoodClassifier: Closing the Interpreter

```
public void close() {
    interpreter.close();
}
```

10.6.11 Explanation

The close() method releases the resources used by the TensorFlow Lite Interpreter after the classification process.

10.7 ResultActivity

10.7.1 Purpose

ResultActivity displays the food classification result. It receives the predicted food label, confidence score, and image information from CameraActivity.

It also displays nutritional information using a predefined local nutrition map.

10.7.2 Important Code Snippet: Receiving Prediction Data

Listing 10.14: ResultActivity: Receiving Classification Result

```
String label = getIntent().getStringExtra(EXTRA_LABEL);

float confidence = getIntent().getFloatExtra(
    EXTRA_CONFIDENCE, 0f
);

String imagePath = getIntent().getStringExtra(
    EXTRA_IMAGE_PATH
);

String imageUri = getIntent().getStringExtra(
    EXTRA_IMAGE_URI
);

if (label == null) {
    label = "Unknown";
}

tvFoodName.setText(capitalize(label));

tvConfidence.setText(
    "Prediction confidence  "
    + Math.round(confidence * 100) + "%"
);
```

10.7.3 Explanation

- getIntent() retrieves the data sent by CameraActivity.
- EXTRA_LABEL contains the predicted food name.
- EXTRA_CONFIDENCE contains the prediction score.
- The confidence value is converted into a percentage.
- The predicted food name and confidence are displayed on screen.

10.7.4 Important Code Snippet: Nutrition Information

Listing 10.15: ResultActivity: Displaying Nutrition Information

```
private static final Map<String, float[]> NUTRITION =
        new HashMap<>();

static {
    NUTRITION.put("orange",
        new float[]{52f, 13f, 1.0f, 0.2f});

    NUTRITION.put("apple",
        new float[]{52f, 14f, 0.3f, 0.2f});

    NUTRITION.put("banana",
        new float[]{89f, 23f, 1.1f, 0.3f});

    NUTRITION.put("pizza",
        new float[]{266f, 33f, 11f, 10f});

    NUTRITION.put("burger",
        new float[]{295f, 24f, 17f, 14f});
}

float[] n = NUTRITION.get(
    label.toLowerCase(Locale.ROOT)
);

if (n != null) {
    tvCalories.setText(fmt(n[0]) + " kcal");
    tvCarbs.setText(fmt(n[1]) + " g");
    tvProtein.setText(fmt(n[2]) + " g");
    tvFat.setText(fmt(n[3]) + " g");
} else {
    tvCalories.setText("--");
    tvCarbs.setText("--");
    tvProtein.setText("--");
    tvFat.setText("--");
}
```

10.7.5 Explanation

- A HashMap stores predefined nutritional values for food items.
- Each array contains calories, carbohydrates, protein, and fat per 100 grams.
- The predicted food label is used to search the nutrition map.
- If the food is found, its nutritional values are displayed.
- If no matching entry exists, the application displays placeholder values.

Note: Nutritional values are predefined reference values in the application. They are not generated by the AI model.

10.8 Overall Application Workflow

The important source code sections work together to complete the FoodAI application workflow:

1. SplashActivity displays the splash screen.
2. LoginActivity provides login and sign-up functionality.
3. MainActivity provides access to the food scanning feature.
4. CameraActivity captures an image or receives an image from the gallery.
5. FoodClassifier processes the image using TensorFlow Lite.
6. ResultActivity displays the predicted food label, confidence, and available nutritional information.

10.9 Code Documentation Summary

The source code is organized into separate Java classes to make the application easier to understand and maintain. Each class performs a specific responsibility, from user interaction to machine learning inference and result presentation.

The code snippets included in this chapter represent important parts of the actual FoodAI source code. The complete Java source files are maintained in the project repository.

11 Expected Learning Outcomes

This practical project supports the following learning outcomes.

11.1 Understanding Android Studio

The project provides practical experience with the Android Studio development environment, including project opening, source code organization, Gradle configuration, and application building.

11.2 Cloning and Inspecting GitHub Repositories

The reference repository provides an opportunity to understand how an existing Android project is cloned, opened, and inspected.

The student can identify important project files and understand how the application is organized.

11.3 Integrating AI and Machine Learning Models

The FoodAI project introduces the process of integrating a TensorFlow Lite model into an Android application.

It demonstrates the role of model files, input preparation, inference, and output interpretation.

11.4 Understanding Image Processing

The project explains how an image must be prepared before it can be passed to a machine learning model.

Important concepts include image resizing, pixel conversion, and input normalization.

11.5 Understanding TensorFlow Lite

The project provides practical exposure to TensorFlow Lite and its role in executing machine learning models on mobile devices.

11.6 Debugging and Testing

The development process helps students understand how to inspect build output, verify configuration, test application components, and identify implementation issues.

11.7 GitHub Repository Management

The project demonstrates how source code and project documentation can be stored and maintained in a GitHub repository.

11.8 Implementation Explanation

The project report and source code documentation help the student explain the application structure, development tools, model integration, and testing process.

12 README and Repository Documentation

12.1 Purpose of README

A README file is an important part of a software project. It provides an overview of the application and explains how another person can understand, install, and run it.

The FoodAI repository should contain a README file in its root directory.

12.2 Required README Contents

12.3 Repository Organization

The final GitHub repository should contain all necessary source code, Gradle configuration, application resources, model assets, and documentation.

Temporary files and unnecessary generated build files should be excluded where appropriate.

The repository should be checked before submission to ensure that the required files are acces-

sible.

13 Limitations and Future Improvements

13.1 Project Limitations

The final performance of FoodAI depends on the correct implementation of its Android components and compatibility of the TensorFlow Lite model.

The following areas require verification:

- Successful application build and installation.
- Correct camera permission handling.
- Camera preview and image capture.
- Model loading and compatibility.
- Correct image preprocessing.
- Correct output tensor interpretation.
- Correct label mapping.
- Actual food classification results.

13.2 Future Improvements

Possible future improvements include:

- Supporting additional food categories.
- Improving image preprocessing.
- Improving the user interface.
- Adding better error handling.
- Improving classification performance.
- Adding more test images.
- Improving application responsiveness.
- Providing more detailed prediction information.

14 Conclusion

FoodAI is an AI-based Android mini-project organized using Android Studio and TensorFlow Lite-related components.

This practical report explains the project objectives, development tools, instructor's reference repository, GitHub repository, Android Studio project structure, application components, and intended classification workflow.

The project provides an opportunity to understand Android application development, camera integration, image processing, machine learning model integration, repository management, and application testing.

The final implementation should be verified by building and running the FoodAI application, testing its image input process, and confirming that the model produces the expected classification output.

The practical submission includes the complete source code, GitHub repository, README documentation, code snippets PDF, practical report PDF, and zipped Android Studio project.

15 References and Acknowledgements

1. Instructor's reference repository:

   https://github.com/Uzair390-Del/NumberGuessingAppTflite

2. Student's FoodAI repository:

   https://github.com/safa699/FoodAI

3. Android Developers Documentation:

   https://developer.android.com/

4. TensorFlow Lite Documentation:

   https://www.tensorflow.org/lite

5. CameraX Documentation:

   https://developer.android.com/media/camera/camerax

6. GitHub Documentation:

   https://docs.github.com/